

Assignment No 2

Aim : Implement Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms in Python

Theory : Breadth-First Search (BFS)

Breadth-First Search (BFS) is one of the fundamental graph traversal algorithms used to visit all the vertices of a graph systematically. The algorithm starts from a selected source vertex and explores all of its neighboring vertices before moving to the next level of vertices. In other words, BFS traverses a graph level by level. It is particularly useful when the shortest path between two vertices in an unweighted graph needs to be determined. BFS uses a queue data structure, which follows the First-In-First-Out (FIFO) principle, to maintain the order of visiting nodes.

The working of BFS begins by selecting a starting vertex and marking it as visited. The starting vertex is then inserted into the queue. The algorithm repeatedly removes the front element from the queue, visits it, and inserts all of its unvisited neighboring vertices into the queue. These newly added vertices are also marked as visited to avoid revisiting them and entering an infinite loop. The process continues until the queue becomes empty, indicating that all reachable vertices have been explored.

The BFS algorithm guarantees that vertices are visited according to their distance from the source vertex. First, all vertices at distance one are visited, followed by vertices at distance two, and so on. Due to this property, BFS is widely used in applications such as finding the shortest path in unweighted graphs, social networking systems to determine connections between users, web crawlers, network broadcasting, GPS navigation systems, and peer-to-peer communication networks.

Consider a graph where vertex **A** is connected to vertices **B** and **C**, vertex **B** is connected to **D** and **E**, and vertex **C** is connected to **F**. If BFS starts from vertex **A**, it first visits **A**, then visits **B** and **C**, followed by **D**, **E**, and **F**. Thus, the traversal order becomes:

A → B → C → D → E → F

The time complexity of BFS is $O(V + E)$, where **V** represents the number of vertices and **E** represents the number of edges in the graph. The space complexity is $O(V)$ because the algorithm maintains a queue and a set of visited vertices.

Although BFS is highly efficient for finding shortest paths and level-order traversal, it requires more memory because all neighboring vertices at each level must be stored in the queue. Therefore, for very large graphs, the memory requirement may become significant.

Depth-First Search (DFS)

Depth-First Search (DFS) is another important graph traversal algorithm that explores a graph by moving as deeply as possible along a branch before backtracking. Unlike BFS, which traverses vertices level by level, DFS follows a path from the starting vertex to the deepest reachable vertex and then returns to explore unexplored branches. DFS can be implemented using recursion or by explicitly using a stack data structure.

The DFS algorithm begins by selecting a source vertex and marking it as visited. It then recursively visits one of its unvisited neighbors and continues this process until it reaches a vertex that has no

unvisited neighbors. At this point, the algorithm backtracks to the previous vertex and explores any remaining unvisited neighbors. This process continues until all vertices in the graph have been visited.

DFS is particularly useful in applications where all possible paths need to be explored. It is widely used in solving puzzles and mazes, detecting cycles in graphs, topological sorting, path finding, connected component analysis, dependency resolution, and artificial intelligence search techniques. Since DFS explores one path completely before exploring others, it requires less memory than BFS in many situations.

For example, consider the same graph where vertex **A** is connected to **B** and **C**, vertex **B** is connected to **D** and **E**, and vertex **C** is connected to **F**. Starting DFS from vertex **A**, the algorithm visits **A**, then moves to **B**, continues to **D**, backtracks to **B**, visits **E**, moves to **F**, and finally returns to visit **C**. The traversal order becomes:

A → B → D → E → F → C

DFS can be implemented recursively because recursion naturally maintains the stack of vertices. Alternatively, an explicit stack data structure can be used to perform iterative DFS. The algorithm ensures that every vertex and edge is processed exactly once.

The time complexity of DFS is $O(V + E)$, where V is the number of vertices and E is the number of edges. The space complexity is $O(V)$ due to the recursion stack or explicit stack used during traversal.

One of the major advantages of DFS is its lower memory consumption compared to BFS when dealing with deep graphs. It is also well suited for problems involving exhaustive search. However, DFS does not guarantee the shortest path in an unweighted graph, and deep recursion may lead to stack overflow in very large graphs.

Difference Between BFS and DFS

Breadth-First Search and Depth-First Search differ mainly in their traversal strategies and data structures. BFS explores nodes level by level using a queue and guarantees the shortest path in unweighted graphs. In contrast, DFS explores nodes deeply using recursion or a stack and is more suitable for exhaustive searches. BFS generally consumes more memory because it stores all vertices at a given level, while DFS often requires less memory but may not find the shortest path. BFS is commonly used in routing, shortest-path problems, and broadcasting systems, whereas DFS is preferred for cycle detection, topological sorting, maze solving, and connectivity analysis.

Both algorithms are fundamental techniques in graph theory and form the basis for many advanced algorithms used in computer science, networking, artificial intelligence, and data analysis.

Python Code : Here is a Python implementation of both Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms using an adjacency list representation of a graph.

Conclusion :

In this practical, Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms were implemented successfully in Python. BFS traverses the graph level by level using a queue, whereas DFS explores nodes deeply using recursion or a stack. The implementation helped in understanding graph traversal techniques and their applications in solving real-world problems.

Use **BFS** when finding the **shortest path** in a simple map is needed

Use **DFS** for problems requiring you to test **every possible choice**